

B: page784 实现和构建

本文由总览原第 5、7.1、7.3 节拆出，作为 B 的独立实现、构建和专项验收手册。全局基线、Qwen layer 命中、三人调用关系、服务冒烟和合并标准见 [Qwen3.5 优化分工总览](#)。

1. 实现

1.1 实现范围

B 独立负责 `rocm_aiter_unified_attention_page784.py` 的完整执行路径，不与 A 共享函数体。A 的宿主 gate 已保证 gfx936、Q24/KV4/D256、BF16、单请求 prefill、page shape，以及 query/output 连续等公共前置条件。K/V cache 允许比赛 hybrid allocator 产生的交错 view，地址由运行时 stride 决定。进入 B 的 `page784.prefill` 后，再检查 page784 特有条件：

```
current K/V 存在, query_len >= 128, context_len >= 784
query_len <= 4096, packed_pages <= 160
单请求, num_actual_tokens == query_len
```

任一 page784 条件不满足时返回 `False`，由 A 的路由继续 GQA6。命中时，B 依次完成 residual pack、main/residual 两次官方 FlashAttention、LSE merge，并返回 `True`。

page784 的精简实现

cheatsheet 将 page784 描述为 `768 main + 16 tail/boundary + current` 三部分。本分支为了 缩短代码，把后两部分合并成一个 residual：

```
main      = 每个完整 page 的前 768 token, B 调用 non-causal FlashAttention
residual  = 每页 16-token tail + boundary token + current K/V, B 打包后调用 causal FA
output    = B 用 main_lse/residual_lse 在 FP32 中稳定合并
```

当前 fast path 要求 `query_len >= 128`、`context_len >= 784`、`query_len <= 4096`、`packed_pages <= 160`、单请求且 `num_actual_tokens == query_len`。不满足时返回 `False`，随后使用 A 的 GQA6；只有更外层宿主 gate 不满足时才回官方 AITER。

`_pack_page784` 分别接收 K/V cache 和 current K/V 的完整运行时 stride。不得再从 `(784, 4, 256)` shape 推导 packed offset，也不得用仅连续 cache 的小矩阵替代交错 view 测试。

workspace 和 page metadata 可以缓存，但 key 必须包含 device/dtype 以及会改变 metadata 内容的长度。不得在每层、每 token 重建大 tensor。

这属于预防性显存上界，不是 OOM 异常恢复：当前两组 token buffer 与两组 page160 buffer 合计约 136 MiB，首次分配失败时不会捕获 `torch.OutOfMemoryError`。cheatsheet 的 page96 停止线约占 120 MiB；若改为 96，B 必须先确认所有目标 4K--32K 请求仍能命中。

B 同时维护 pack layout、两次 FlashAttention 的长度/block table 契约和 LSE merge 数学。这些步骤属于一条执行路径，放在同一文件并由一人维护，避免跨负责人约定 residual layout。

B 与 A/C 的协作和解耦

- **B给A的唯一运行时接口**是 `page784.prefill(...) -> bool`。所有返回 `False` 的条件必须在 `pack/FA/merge` 启动前判定，保证A可直接用原output执行GQA6；返回 `True` 时B必须已经完成 两次FA和LSE merge，A不得追加任何 attention 计算。
- **公共gate与专用gate分工**：A证明 `gfx936`、`Q24/KV4/D256`、`BF16`、单请求 `prefill`、`cache page shape` 和 `query/output layout` 等公共条件；B只检查 `query_len/context_len/packed_pages`、`current K/V` 和实际 token 数等 `page784` 条件。若B开始依赖新的公共前置条件，必须先让A把它 加入宿主 gate，不能在B内部默认为真。
- **交付给A的验收结果**：B独立提供 `page784` 命中时与官方FA/GQA结果的同输入误差、边界 `fallback` 结果和 `workspace` 峰值；A据此验收路由。B文件不导入GQA6，也不修改 `attention backend`，因此 `page784` 内部优化可独立消融。
- **B和C的边界**：没有直接函数、`workspace` 或 `native ABI` 依赖。`gfx936` 判断由A的公共 `attention gate` 调用，B只是间接接受保护；C修改 `_rocm_C` 不要求B重编FA，B修改 `Triton` 也不要求C改HIP kernel。二者只在最终新wheel、同一服务请求和显存停止线上联合验收。
- **依赖边界**：B调用比赛容器预装的DCU `flash_attn_2_cuda`，只维护调用和数据准备；不维护FA源码、不从PyPI 下载wheel、也不因 `page784` 代码变化重编FA。

1.2 以官方源码为模板的改法

B 的实现应从官方 `metadata`、`paged-cache` 地址方式和 `cascade attention` 开始，不自行定义 另一套 `metadata` 或 `attention` 状态：

- 官方 `FlashAttentionMetadata`
- 官方 `FlashAttention cascade` 与 `LSE merge`
- 官方 `Triton paged attention`

page784：按官方 `metadata/block table` 准备输入

1. 直接使用官方 `FlashAttentionMetadata` 的 `query starts`、`sequence lengths`、`num_actual_tokens` 和 `block table`；不建立并行 `host metadata` 类。
2. `_pack_page784` 只把不规则的 16-token tail、`boundary` 和 `current K/V` 按逻辑顺序 变成 `page64`，输出直接交给同文件的 `residual FlashAttention` 调用。
3. `main` 的 `block table` 复用官方表的完整-page 前缀；`residual table` 使用缓存的连续 `page64` 编号。长度 `tensor` 与 `workspace` 按 `device/dtype/长度` 缓存，不能每层重建。
4. `cache layout` 地址计算照搬官方 `Triton paged-attention` 的 `runtime stride` 思路；`K/V` 和 `current K/V` 各自传递 `stride`，不能恢复为定长连续 `offset`。
5. 以官方 `cascade_attention` 为控制流模板：`main` 使用 `non-causal paged varlen_fwd_unified`，`residual` 使用 `causal` 调用，两次都返回 `FP32 LSE`。
6. `_merge_page784` 的 `max-LSE` 权重必须逐项对应官方 `merge_attn_states`；当前保留 `modular` 性能版本的四行/CTA 特化。

因此 B 负责的是一条封闭的 `page784` 路径：`gate -> pack -> two FA -> merge -> bool`。

1.3 B 的代码索引

Path	当前行号	内容介绍	可参考的官方代码（fa718036）
<code>vllm/v1/attention/ops/rocm_aiter_unified_attention_page784.py</code>	1-70	文件调用链、 <code>workspace cache</code> 与 <code>_pack_page784</code> ；按真实 <code>stride</code> 打包 <code>tail</code> 、 <code>boundary</code> 和 <code>current K/V</code>	<code>vllm/v1/attention/ops/triton_unified_attention.py</code> 的 <code>block-table/stride</code> 处理

Path	当前行号	内容介绍	可参考的官方代码（ fa718036 ）
同上	73-97	<code>_merge_page784</code> ：按两组 LSE 稳定合并 main/residual 输出	<code>vllm/v1/attention/backends/flash_attn.py::cascade_attention</code> 与 <code>vllm/v1/attention/ops/merge_attn_states.py</code>
同上	100-187	workspace/metadata 缓存、page784 边界 gate、main/residual 输入准备	<code>vllm/v1/attention/backends/flash_attn.py::FlashAttentionMetadata</code>
同上	191-230	两次官方 FlashAttention、LSE 规格化、merge launch 和 <code>True</code> 返回	<code>vllm/v1/attention/backends/flash_attn.py::cascade_attention</code>

1.4 B 的验收

- page784 边界至少覆盖 `query_len=127/128/4096/4097`、`context_len=783/784`、`packed_pages=160/161`。
- 检查 pack 后 token 顺序、main/residual lengths 和 block table；B 独立保证完整输出与 官方 attention 对齐。
- contiguous cache 和交错 cache 都必须命中并与官方 FA 对齐；验证记录必须打印实际 stride，不能只检查 shape。
- 三档固定请求（4-8K、8-16K、16-32K）必须完整文本精度通过；不能只做 kernel allclose。

2. 构建与专项测试

2.1 完整 wheel 构建与隔离安装

B 使用下面这组已经在当前比赛容器执行成功的命令。构建放在临时 worktree，是因为 OpenDAS `setup.py` 会重写源码树中的 `vllm/version.py`；`--target` 安装不会覆盖容器依赖。

```
cd /public/home/tangyu408/Qwen_DCU_Worker_0/pr2026-bh408-gqa-page784-k5120
git worktree add --detach /tmp/qwen35-build-source HEAD
cd /tmp/qwen35-build-source
mkdir -p /tmp/qwen35-build/{dist,bdist,cache}

VLLM_TARGET_DEVICE=rocm MAX_JOBS=16 python3 setup.py \
  build --build-base /tmp/qwen35-build/build \
  bdist_wheel --bdist-dir /tmp/qwen35-build/bdist \
  --dist-dir /tmp/qwen35-build/dist

python3 -m pip install --no-deps --target /tmp/qwen35-build/site \
  /tmp/qwen35-build/dist/vllm-*.whl
```

后续命令统一使用：

```
REPO_ROOT=/public/home/tangyu408/Qwen_DCU_Worker_0/pr2026-bh408-gqa-page784-k5120
ARTIFACT_ROOT=/tmp/qwen35-build
SITE_DIR=/tmp/qwen35-build/site
CACHE_ROOT=/tmp/qwen35-build/cache
cd "$REPO_ROOT"
```

2.2 page784 专项检查

B 的自定义 pack/merge 是 Triton JIT，两次 attention 使用环境提供的已编译 `flash_attn_2_cuda`。先确认比赛环境的 DCU FlashAttention wheel 和目标入口存在：

```
PYTHONPATH="$SITE_DIR" python3 -c 'import importlib.metadata as m; \
import flash_attn_2_cuda; from flash_attn import varlen_fwd_unified; \
print(m.version("flash-attn"), varlen_fwd_unified.__module__)'
```

```
ruff check vllm/v1/attention/ops/rocm_aiter_unified_attention_page784.py
python3 -m py_compile vllm/v1/attention/ops/rocm_aiter_unified_attention_page784.py
mkdir -p "$CACHE_ROOT/b-triton"
CHECK_SCRIPT="$PWD/docs/cscv/verify_qwen35_optimizations.py"
RUN_DIR="$(mktemp -d /tmp/qwen35-b-check.XXXXXX)"
(cd "$RUN_DIR" && HIP_VISIBLE_DEVICES=0 \
    TRITON_CACHE_DIR="$CACHE_ROOT/b-triton" PYTHONPATH="$SITE_DIR" \
    python3 "$CHECK_SCRIPT" page784)
```

非代码步骤：不得用公开 PyPI wheel 覆盖比赛环境的 FlashAttention；只改 page784 Python/ Triton 时无需重编 FA。测试前观察至少约 136 MiB 的额外 workspace 余量，并分别验证 q= 128、context= 784/800 的命中、context800交错 cache命中，以及query/context下界、query上界和workspace页数上界共4个返回 `False` 且输出不变的边界。

3. 联合验证入口

完成本文件的专项检查后，按 [总览第 7.5 节](#) 继续执行同一新 wheel 的 DP1/DP2 小数据服务冒烟；实测版本、结果、合并顺序和故障备注 统一保留在总览第 7.6--7.8 节。